



Retinal Review Workbench

Deployment and Operations Manual

Workstation setup, hospital Model API, and routine operations

Software version	0.7.0
Manual version	1.1 (2026-09-23)
Document ID	RRW-OPS-UM-001
Audience	Hospital IT, deployment operators, maintainers
Owner	Retinal Review Workbench maintainers
Revision history	1.1 – layout refresh, new architecture figure, readable commands 1.0 – controlled release documentation for v0.7.0
Companion	Clinician User Manual (RRW-CLIN-UM-001)

Research and clinical-support proof of concept for public or synthetic data. Model output is assistive evidence; human review remains authoritative.

Table of contents

1	About This Manual	2
2	Architecture and Supported Scope	2
2.1	Topology	2
2.2	Boundaries	3
2.3	Data ownership	3
3	Installation	3
3.1	Review workstation	3
3.2	Linux server checkout	4
3.3	Configuration	4
4	Model API Setup and Startup	4
4.1	One-time online setup	4
4.2	Normal startup	5
4.3	Health and model metadata	5
4.4	Offline after setup	5
5	Routine Operations	5
5.1	systemd service	5
5.2	Connect a workstation	6
5.3	Daily readiness check	6
6	Security, Privacy, Backup, and Restore	6
6.1	Security and privacy	6
6.2	Backup	6
6.3	Restore	7
6.4	CVAT	7
7	Upgrade, Rollback, and Release Acceptance	7
7.1	Upgrade	7
7.2	Rollback	7
7.3	Troubleshooting	8
7.4	Release acceptance	8

1 About This Manual

This manual covers the supported review workstation and the hospital Model API deployment of Retinal Review Workbench 0.7.0. It is written for hospital IT, deployment operators, and maintainers. Clinical use of the workstation is described in the separate *Clinician User Manual*.

Chapter	Covers
Architecture and supported scope	Topology, component roles, boundaries, data ownership
Installation	Review workstation, Linux server checkout, configuration
Model API setup and startup	One-time online setup, normal startup, health checks, offline behaviour
Routine operations	systemd service, workstation connection, daily readiness check
Security, privacy, backup, and restore	Data handling, secrets, backup scope, restore procedure, CVAT
Upgrade, rollback, and release acceptance	Controlled upgrade, rollback, troubleshooting, acceptance record

Safety – protected deployment

Keep the Model API on a protected hospital network. Do not expose it directly to the public internet. Apply hospital firewall, TLS, authentication, and secret-management policy at the deployment boundary.

2 Architecture and Supported Scope

2.1 Topology

The deployment has three parts: the clinician workstation, the protected hospital LAN, and the hospital GPU host (Figure 2.1).

Component	Role
Review workstation	React/Vite clinician UI and FastAPI review routes at /app/; Workspace catalog, local review state, annotations, and export.
Model API	FastAPI service on a GPU host with APP_PROFILE=model_api and MODEL_RUNTIME=local.
RETFound	Five-class DR grade suggestion through retfound-aptos5.
PRISM-DR	Optional lesion ROI suggestions through prism-dr-5fold.
CVAT Online	Optional dense annotation and geometry correction. Not part of offline acceptance.

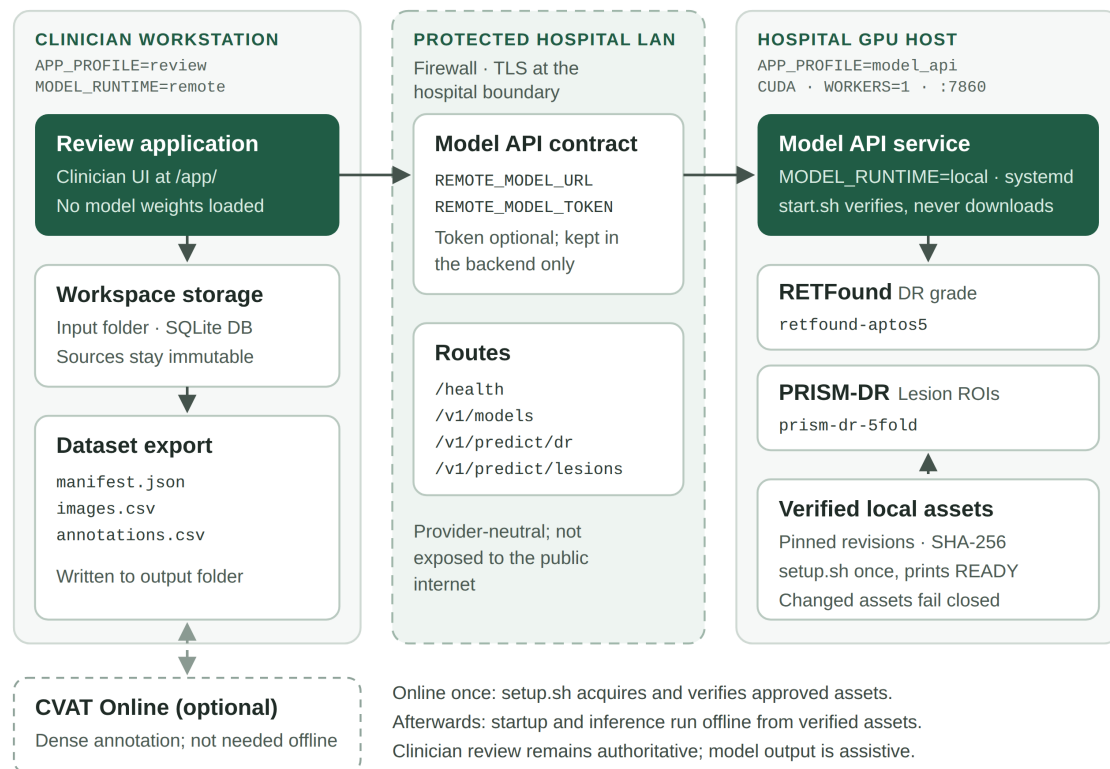


Figure 2.1: Hospital deployment architecture. The review workstation calls the provider-neutral Model API; verified model assets stay on the GPU host.

2.2 Boundaries

The review profile does not load model weights. It calls the provider-neutral Model API. The Model API supports the existing fundus image contracts only. MRI, OCT, PACS, DICOMweb, calibration, XAI/Grad-CAM, and autonomous diagnosis are outside this release.

The application keeps the /ui static surface for compatibility with existing local integrations. Normal users and deployments use /app/; /ui is legacy compatibility only.

2.3 Data ownership

The review workstation owns Workspace configuration, case state, annotations, audit events, and export files. The Model API is stateless for clinical records and returns model metadata, inference results, and provenance. Original admitted source images remain immutable.

3 Installation

3.1 Review workstation

The supported Windows lifecycle is `SETUP.cmd -> START.cmd -> browser`. No machine-wide Administrator access is required. `uv` owns the project-managed Python 3.12 environment; Node.js is taken from a compatible system install or from the official no-admin fallback under `ignored local-state/tools/`.

Steps (Windows).

```
git clone https://github.com/siriponsri/dr-support-screening-poc.git
cd dr-support-screening-poc
SETUP.cmd
START.cmd
```

Expected result. The workstation serves `http://127.0.0.1:8000/app/` and opens it in the default browser. Daily `START.cmd` calls reuse the project environment and frontend dependencies. It rebuilds the frontend only when the tracked source signature or lockfile state requires it; it never runs `npm ci` on a normal start. Use `OPEN_APP.cmd` to reopen the app and `STOP.cmd` to stop only the repository-managed process.

Use `SETUP.cmd --check` for a read-only installation diagnostic and `SETUP.cmd --dev` when test, DICOM, and browser-test extras are required.

3.2 Linux server checkout

Place the release at `/opt/dr-support-screening-poc` when using the included systemd unit. Keep runtime assets in `local-state/` or another private path. Never commit model weights, review databases, or secret environment files.

3.3 Configuration

Use `.env.example` only as a reference. Supply tokens through the private process environment or an approved secret manager. The model-server example is `deployment/model-server.env.example`; `docs/operations/CONFIGURATION.md` lists every variable and its safe default.

4 Model API Setup and Startup

4.1 One-time online setup

Run once on the supported Linux NVIDIA GPU host, with network access:

```
cd /opt/dr-support-screening-poc
./scripts/model-server/setup.sh
```

The setup script creates `.venv`, installs `[models,dicom]`, acquires the pinned RETFound and PRISM-DR sources and checkpoint assets, verifies revisions and SHA-256 values, checks the configured CUDA device, and runs synthetic smoke inference. It prints `READY` only after verification and smoke inference succeed.

Pinned source and asset facts are maintained in `docs/operations/MODEL_SERVER.md` and in:

- `dr_support/providers/retfound.py`
- `dr_support/providers/prism.py`
- `dr_support/providers/prism_assets.json`

4.2 Normal startup

```
cd /opt/dr-support-screening-poc
./scripts/model-server/start.sh
```

Setting	Default
APP_PROFILE / MODEL_RUNTIME	model_api / local
INFERENCE_DEVICE	cuda:0
HOST / PORT	0.0.0.0 / 7860
WORKERS	1

Startup first runs `python -m dr_support.setup_models --model all --verify`. It does not download or silently replace assets.

4.3 Health and model metadata

```
./scripts/model-server/healthcheck.sh
curl --fail http://127.0.0.1:7860/health
curl --fail http://127.0.0.1:7860/v1/models
```

Expected result. /health reports "status":"PASS" with verified assets, and /v1/models lists retfound-aptos5 and prism-dr-5fold.

4.4 Offline after setup

After the one-time setup succeeds, disconnect external network access and restart with `start.sh`. Readiness and local inference use only the verified local source checkouts and checkpoint files. CVAT Online is optional and excluded from offline acceptance.

Safety – fail closed

If a source revision, checkpoint hash, CUDA device, or required asset is missing or modified, startup must fail and require operator intervention. Do not bypass verification by switching silently to another model or device.

5 Routine Operations

5.1 systemd service

Install the included service only when the release is at `/opt/dr-support-screening-poc`:

```
cd /opt/dr-support-screening-poc
sudo ./scripts/model-server/install-service.sh
sudo systemctl start dr-support-model-api
sudo systemctl status dr-support-model-api
./scripts/model-server/healthcheck.sh
```

The service runs as user `drsupport`, reads `/etc/dr-support/model-server.env`, uses one worker, and starts `scripts/model-server/start.sh`.

Lifecycle and logs:

```
sudo systemctl stop dr-support-model-api
sudo systemctl restart dr-support-model-api
sudo journalctl -u dr-support-model-api -n 100 --no-pager
```

5.2 Connect a workstation

Set `REMOTE_MODEL_URL` on the review workstation to the Model API base URL, for example:

```
REMOTE_MODEL_URL=http://model-host.example:7860
```

Use HTTPS across any untrusted segment. When required, set `REMOTE_MODEL_TOKEN` through a private environment or secret manager. The token is never returned to the UI, stored in SQLite, or logged.

5.3 Daily readiness check

1. Confirm the service is active with `systemctl status`.
2. Run `healthcheck.sh` and confirm `/health` reports `PASS`.
3. Confirm `/v1/models` returns both expected model IDs with verified status.
4. Open the review workstation and check **Models & Audit**.
5. Run a public or synthetic smoke image only when the release acceptance procedure requires it.

6 Security, Privacy, Backup, and Restore

6.1 Security and privacy

Use approved public, synthetic, or authorized clinical data. Keep PHI, private DICOM headers, model weights, tokens, passwords, CVAT credentials, runtime SQLite files, and `local-state/` directories out of Git and screenshots. Original source images are immutable.

The product is a research and clinical-support proof of concept. Human review remains authoritative. No product claim establishes regulatory approval, autonomous diagnosis, referral, treatment, or calibrated probability.

6.2 Backup

Back up, according to hospital policy:

- the active Workspace SQLite database;
- approved export packages;
- Workspace configuration;
- the exact release and configuration record.

Keep model assets with their verified hashes and preserve the relationship between release, model assets, and exports. Do not copy secret files into the repository; manage `/etc/dr-support/model-server.env` through the hospital secret-management process.

6.3 Restore

1. Stop the affected service or workstation process.
2. Restore the Workspace database and approved export package to a controlled path.
3. Restore the same release or a validated compatible release.
4. Verify file ownership and permissions.
5. Run the model asset verification and `/health` checks.
6. Open the workstation, inspect the Workspace, and confirm source and annotation hashes before resuming review.

6.4 CVAT

CVAT Online is optional. Keep `CVAT_TOKEN` only in secure runtime configuration. Do not paste it into commands, browser JavaScript, screenshots, documentation, or issue reports. Ambiguous remote writes require manual reconciliation; never retry blindly.

7 Upgrade, Rollback, and Release Acceptance

7.1 Upgrade

1. Keep the previous validated release and model asset directory intact.
2. Place the new clean checkout at a separate path.
3. Run the one-time model setup on the new release if its verified assets are not already present.
4. Run backend tests, Ruff, frontend tests, typecheck, and the production build.
5. Start the new Model API, run the health and model metadata checks, and perform one public or synthetic smoke inference.
6. Connect a review workstation and verify `/app/`, dataset export, and **Models & Audit**.
7. Record the release SHA, software version, model asset hashes, and acceptance result before switching the service configuration.

7.2 Rollback

Stop the service, restore the previous release and configuration, verify the previous asset hashes, and restart. Do not overwrite verified model assets in place. Preserve the failed release for investigation according to hospital policy.

7.3 Troubleshooting

Observation	Action
READY is not printed	Read the setup error. Check Python, dependencies, CUDA, source revisions, checkpoint hashes, and storage.
Startup fails verification	Treat changed or missing assets as a deliberate fail-closed state. Reacquire through the controlled online setup.
/health is not PASS	Check service logs, CUDA visibility, environment values, port binding, and asset verification.
/v1/models lacks a model	Check local assets and the configured provider package. Do not substitute a different model.
Workstation cannot infer	Check REMOTE_MODEL_URL, token handling, firewall policy, and Model API health. Human review and export remain available.
DICOM is unsupported	A non-fundus DICOM is reported as Unsupported modality . Do not expand support to MRI, OCT, PACS, or DICOMweb.

7.4 Release acceptance

Record the release SHA and confirm:

- clean-clone installation succeeds;
- the frontend production build succeeds;
- the review workstation opens at /app/;
- one-time model setup verifies pinned sources and prints READY;
- normal startup is offline-safe;
- /health and /v1/models pass;
- one public or synthetic image completes the supported inference path;
- dataset export contains the documented manifest and CSV files;
- no secrets, PHI, model weights, or runtime databases are tracked;
- the clinician and operator manuals render successfully.